# WA_Sim

```r
if (!require("spatstat.Knet")) {
  # Install the latest version of the package
  install.packages("spatstat.Knet_1.11-0.tar.gz", repos = NULL, type = "source")
  if (!require("spatstat.Knet"))
    stop("Unable to install package")
} else requireversion(spatstat.Knet, "1.11-0")
```

```
## Loading required package: spatstat.Knet

## Loading required package: spatstat.data

## Loading required package: spatstat.sparse

## Loading required package: Matrix

## Loading required package: abind

## Loading required package: tensor

## Loading required package: spatstat.univar

## spatstat.univar 3.1-4

## Loading required package: spatstat.geom

## spatstat.geom 3.5-0

## Loading required package: spatstat.random

## spatstat.random 3.4-1

## Loading required package: spatstat.explore

## Loading required package: nlme

## spatstat.explore 3.5-2

## Loading required package: spatstat.model

## Loading required package: rpart
```

```
## spatstat.model 3.4-0

## Loading required package: spatstat.linnet

## spatstat.linnet 3.3-1

## Loading required package: spatstat

##
## spatstat 3.4-0
## For an introduction to spatstat, type 'beginner'

## spatstat.Knet 3.1-2
```

```r
library(spatstat.geom)
library(spatstat.linnet)
library(spatstat)
library(sf)        #
```

```
## Linking to GEOS 3.13.0, GDAL 3.8.5, PROJ 9.5.1; sf_use_s2() is TRUE
```

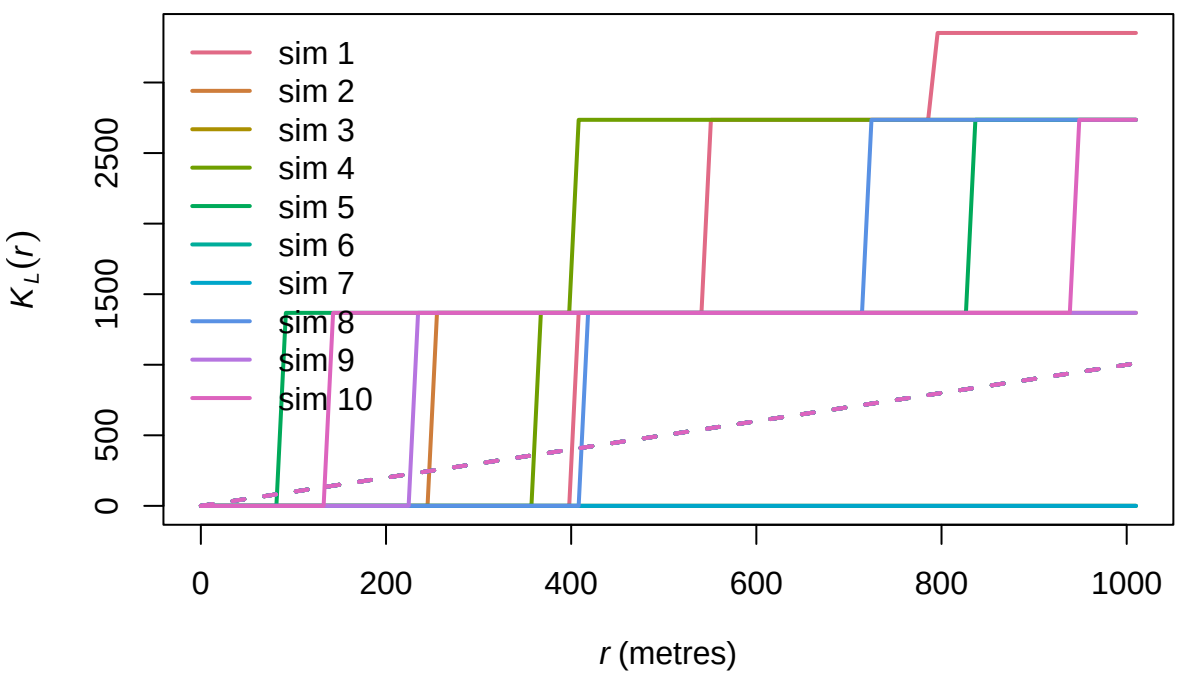
```r
library(dplyr)     #
```

```
##
## Attaching package: 'dplyr'

## The following object is masked from 'package:nlme':
##
##     collapse

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

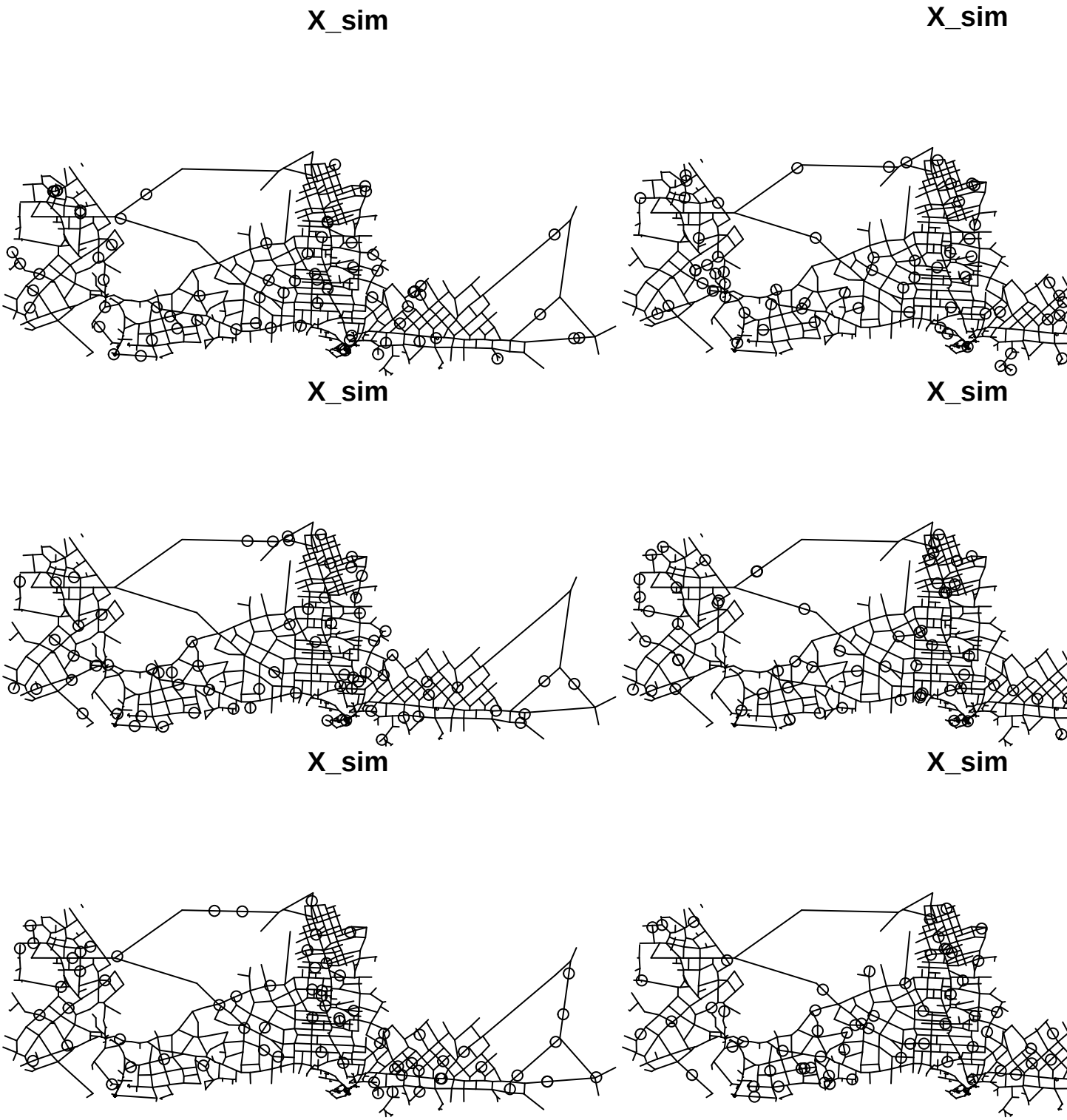
```r
# Data Loading ----------------------------------------------------------------
data("wacrashes", package = "spatstat.Knet")
X_sub1 <- readRDS("Rakshit/WAsub_data/X_sub.rds")
X_sub2 <- readRDS("Rakshit/WAsub_data2/X_sub.rds")
```

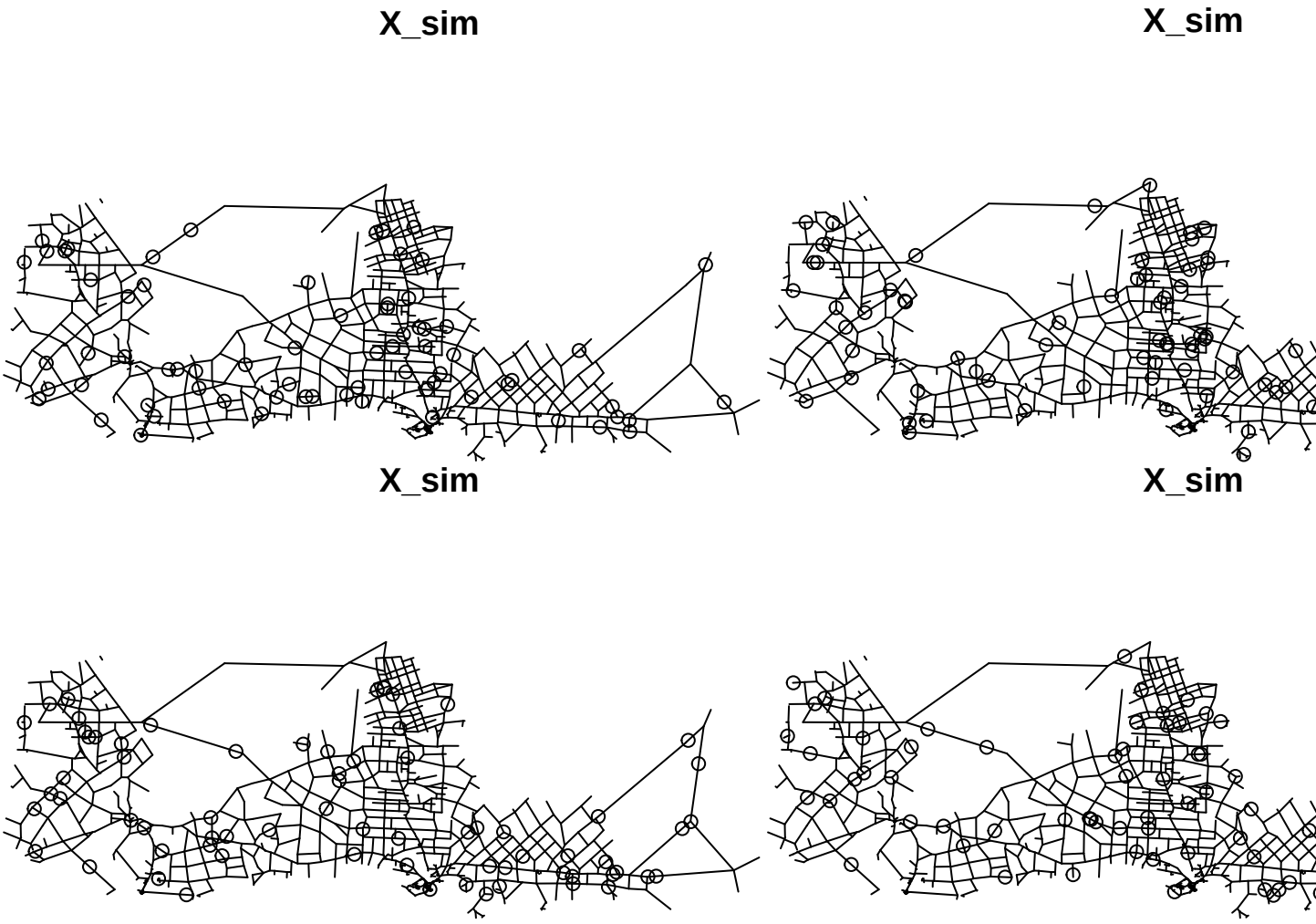
## Original Simulation

```r
L <- domain(X_sub2)       #       lpp
r_grid <- seq(0, 1000, length = 100) # meter

#     i   K
B    <- 10                                #        i
n0   <- 70                               #
cols <- grDevices::hcl.colors(B, "Dark 3")

for (b in seq_len(B)) {
  set.seed(n0 + b)
  X_sim   <- runiflpp(n0, L)              #       n0 * b
  Kb      <- Knet(X_sim, r = r_grid)

  if (b == 1) {
    plot(Kb, legend = FALSE, lwd = 2, col = cols[b], main = "")
  } else {
    plot(Kb, add = TRUE, legend = FALSE, lwd = 2, col = cols[b])
  }
}

legend("topleft",
       legend = paste0("sim ", seq_len(B)),
       col = cols, lwd = 2, bty = "n")
```




```r
for (b in seq_len(B)){
  set.seed(n0 + b)
  X_sim   <- runiflpp(n0, L)              #       n0 * b
  plot(X_sim)
}
```

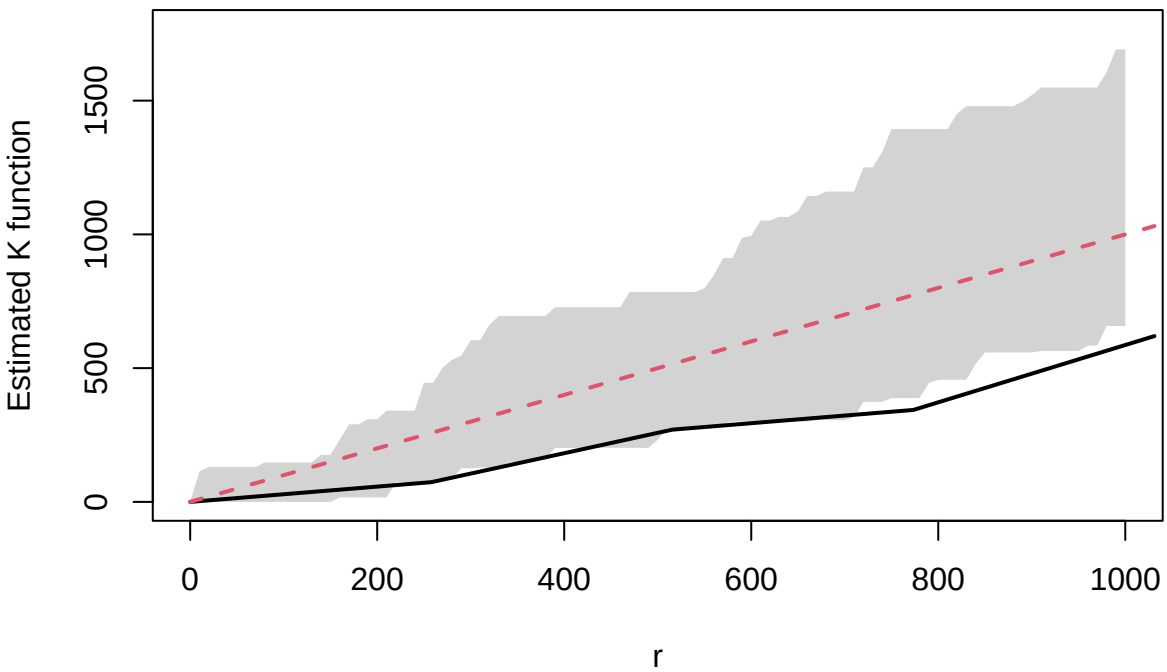

X_sim
X_sim
X_sim
X_sim
X_sim
X_sim




**Confidence Interval**

```
L <- domain(X_sub2)      #      lpp
r_grid <- seq(0, 1000, length = 101) # meter

#    i   K
B    <- 10                              #        i
n0   <- 300                             #
cols <- grDevices::hcl.colors(B, "Dark 3")

for (b in seq_len(B)) {
  X_sim   <- runiflpp(n0, L)            #       n0 * b
  Kb      <- Knet(X_sim, r = r_grid)$est

  if (b == 1) {
    kfuns1 <- c()
    kfuns1 <- cbind(kfuns1,Kb)
    #plot(Kb, legend = FALSE, lwd = 2, col = cols[b], main = "")
  } else {
    kfuns1 <- cbind(kfuns1,Kb)
```

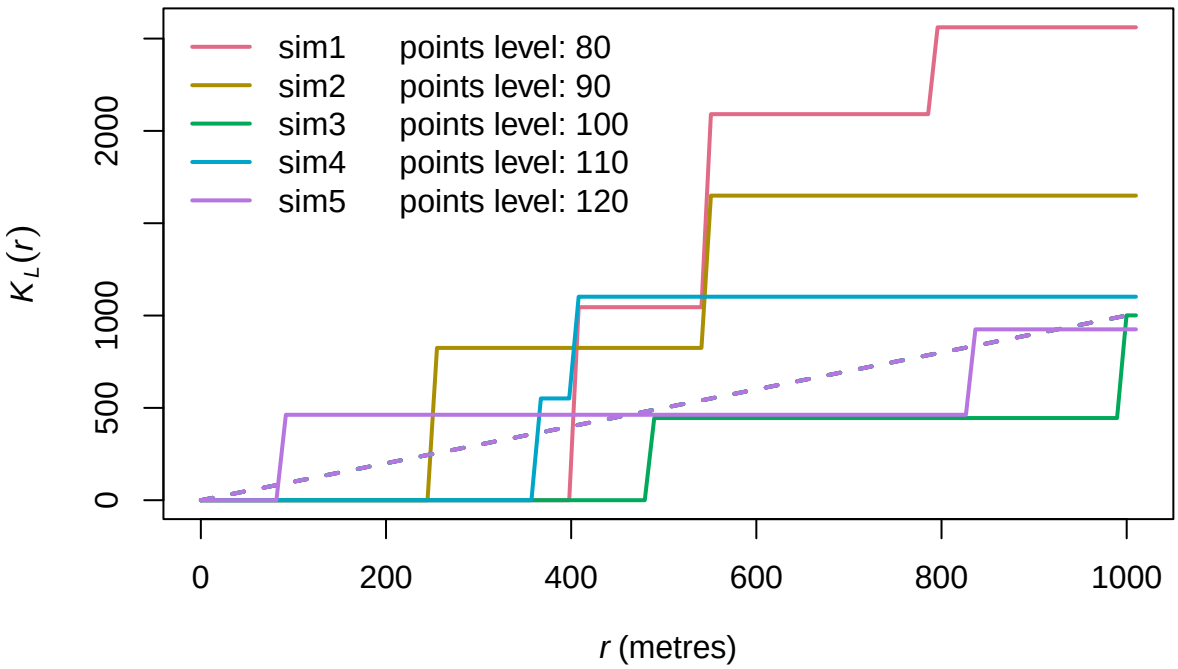
```
    #plot(Kb, add = TRUE, legend = FALSE, lwd = 2, col = cols[b])
  }
}

qts <- apply(kfuns1, 1, quantile, probs = c(0.025, 0.975))


plot(NULL, xlab = "r", ylab = "Estimated K function", xlim = range(r_grid),
ylim = range(kfuns1))

polygon(c(r_grid, rev(r_grid)), c(qts[1, ], rev(qts[2, ])), col = "lightgrey",
border = NA)

plot(Knet(X_sim), add = TRUE, legend = FALSE, lwd = 2)
```

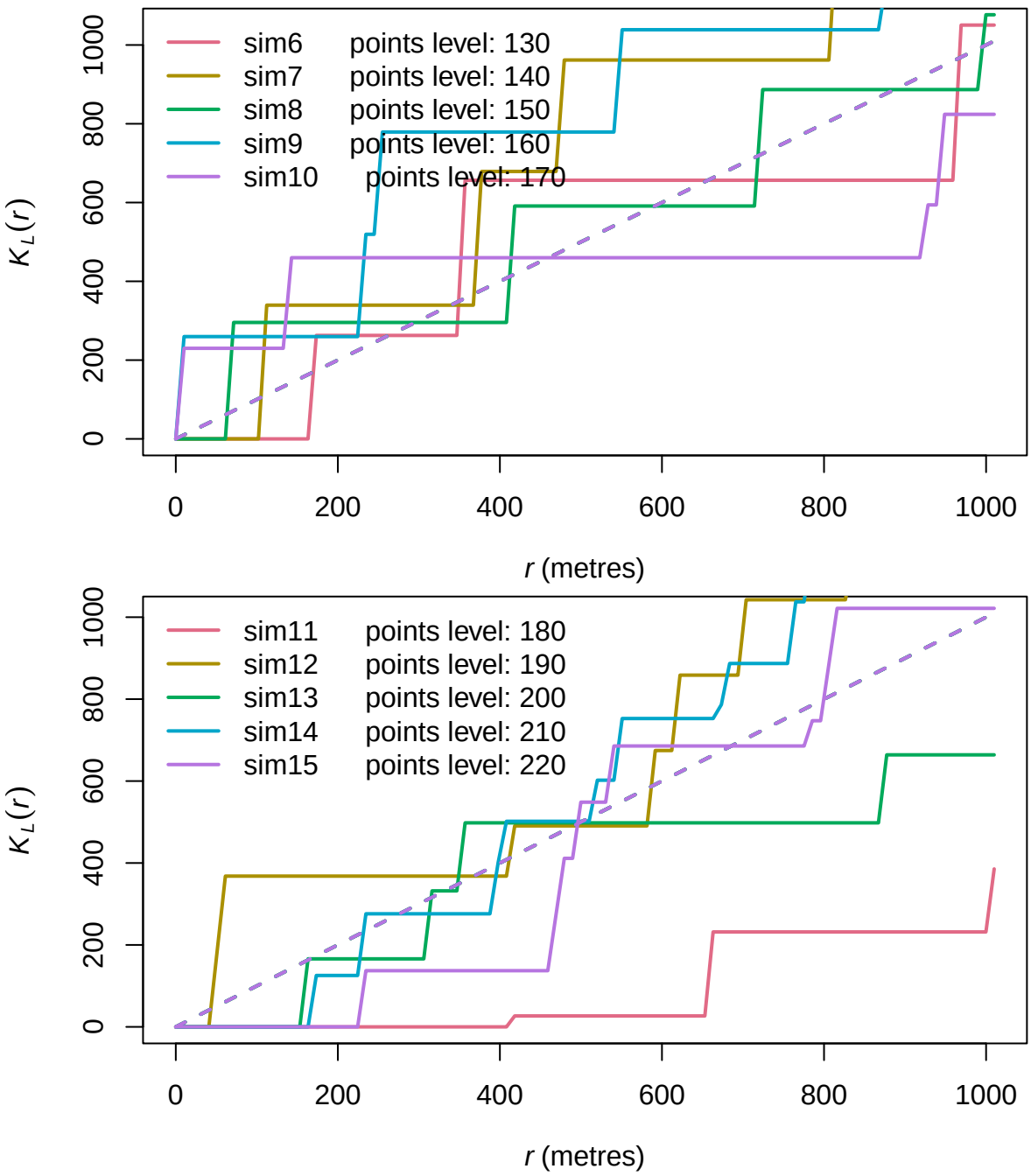


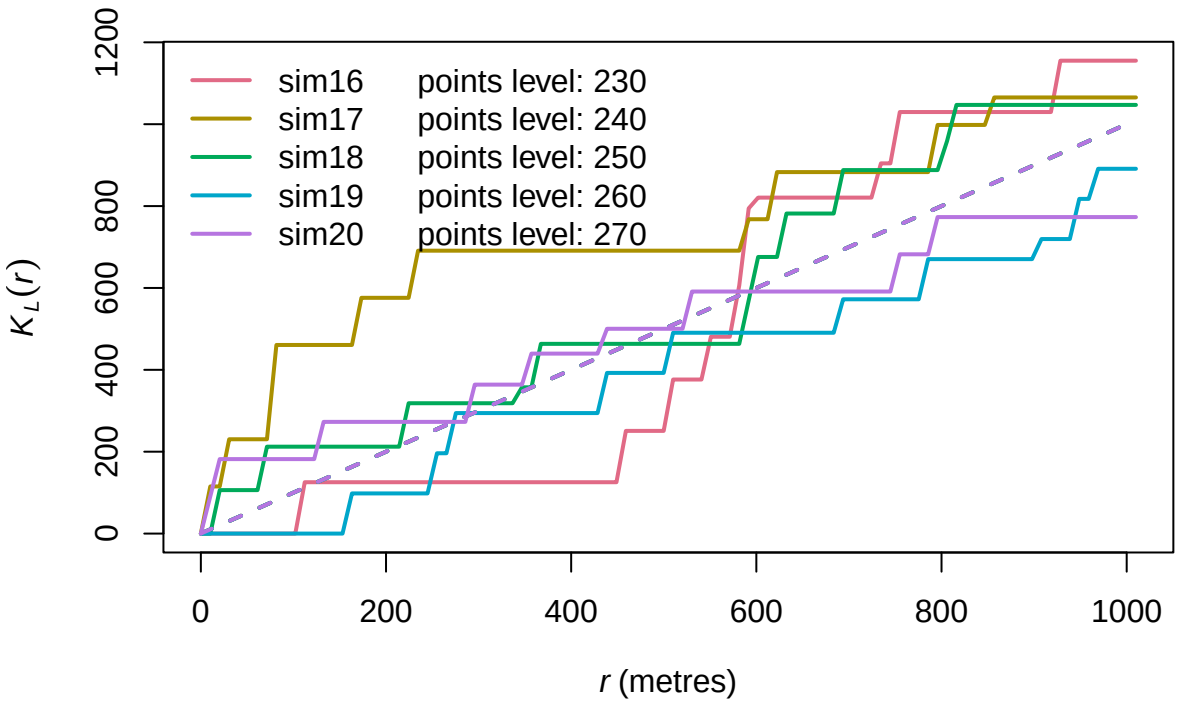

**Increasing the number of Simulating points**

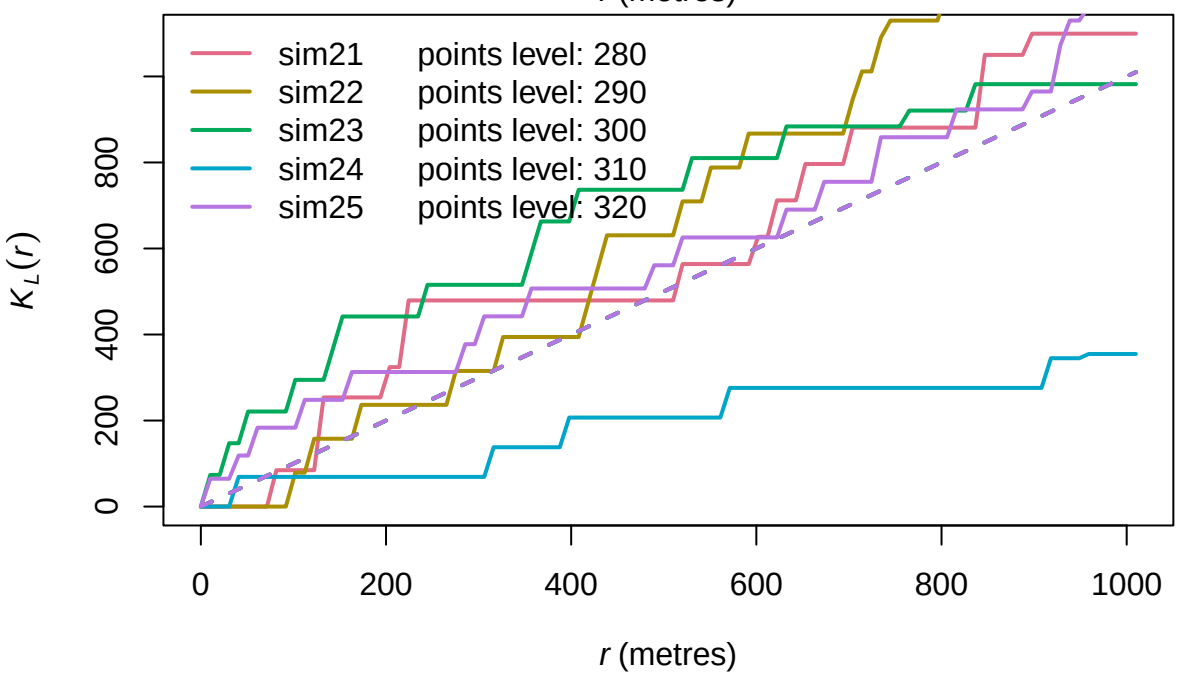
```
L <- domain(X_sub2)      #      lpp
r_grid <- seq(0, 1000, length = 100) # meter

#    i   K
B    <- 50                                #        i
n0   <- 70                               #
cols <- grDevices::hcl.colors(B, "Dark 3")

for (b in seq_len(B)) {
  set.seed(n0 + b)
  X_sim   <- runiflpp(n0 + (10* b), L)              #       n0 * b
  Kb      <- Knet(X_sim, r = r_grid)
  if((b - 1) %% 5 == 0){
```

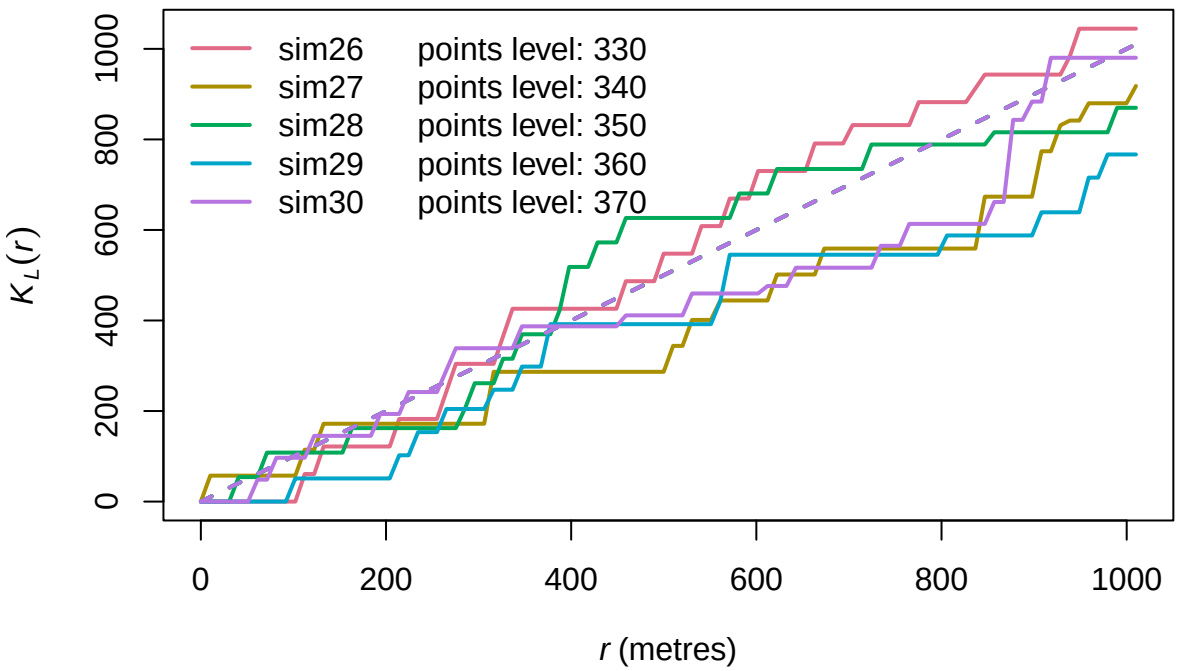
```
    plot(Kb, legend = FALSE, lwd = 2, col = cols[((b - 1) %% 5) * 10 + 1], main = "")
  }else{
    plot(Kb, add = TRUE, legend = FALSE, lwd = 2, col = cols[((b - 1) %% 5) * 10 + 1])
  }
  if (b %% 5 == 0) {
    legend("topleft",
           legend = paste0("sim", (b-4):b,"      points level: ",seq(from = (n0 + (10* (b-4))), to = (n0

           col    = cols[ seq(from = 1, to = 41, by = 10) ],
           lwd    = 2,
           bty    = "n")
  }
}
```

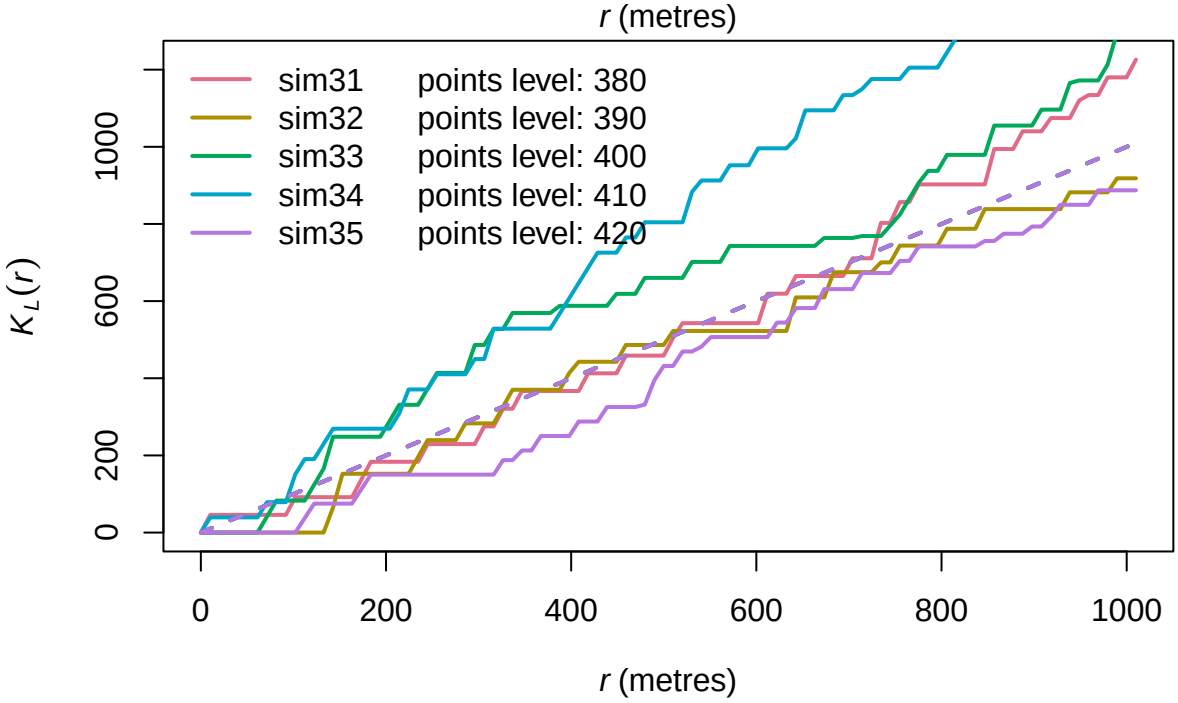


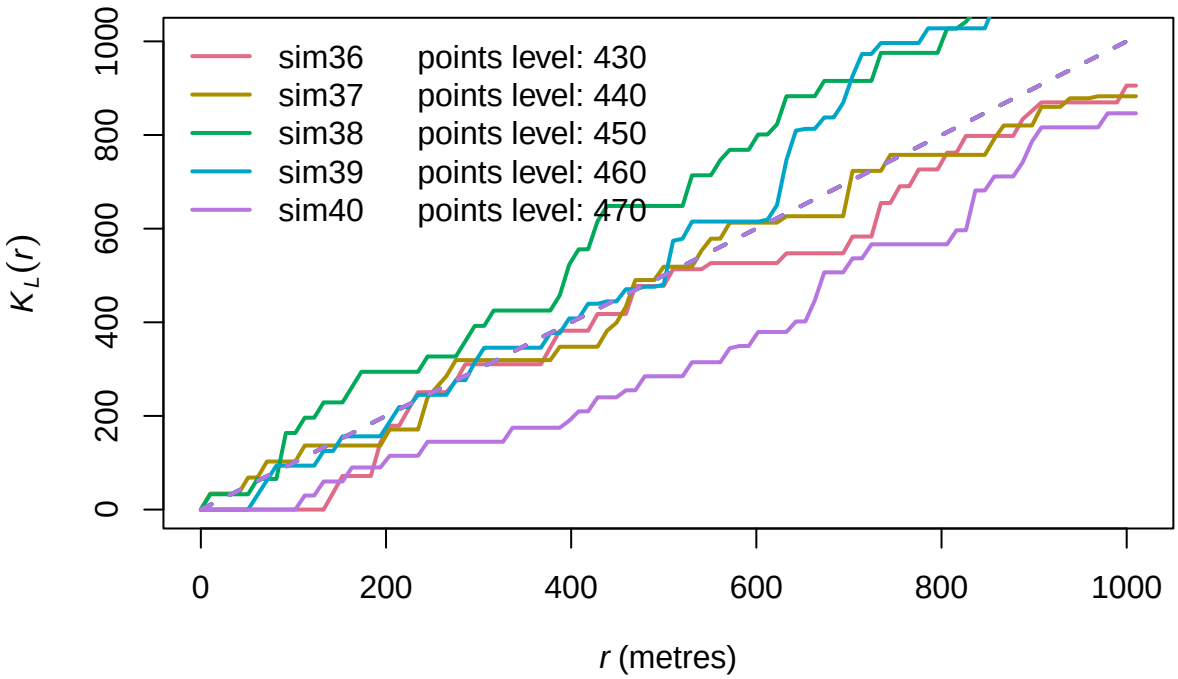

sim6 points level: 130
sim7 points level: 140
sim8 points level: 150
sim9 points level: 160
sim10 points level: 170
$K_L(r)$
0
200
400
600
800
1000
r (metres)
sim11 points level: 180
sim12 points level: 190
sim13 points level: 200
sim14 points level: 210
sim15 points level: 220
$K_L(r)$
0
200
400
600
800
1000
r (metres)

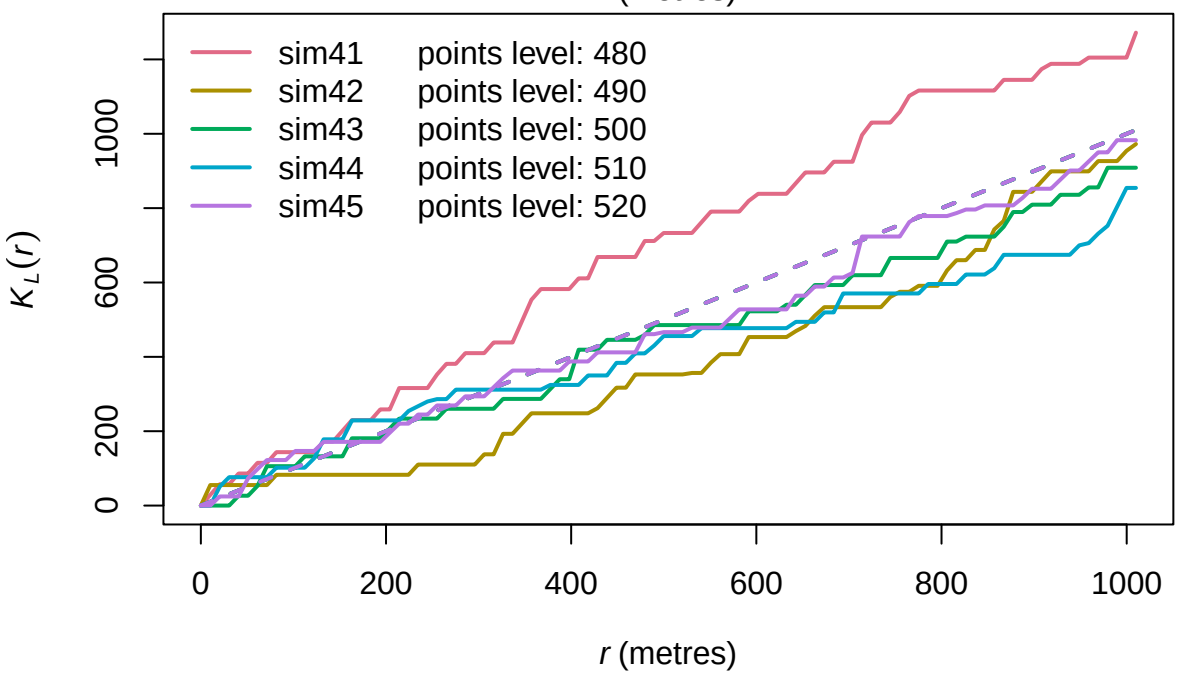

sim16 points level: 230
sim17 points level: 240
sim18 points level: 250
sim19 points level: 260
sim20 points level: 270
$K_L(r)$
0
200
400
600
800
1200
0
200
400
600
800
1000
$r$ (metres)

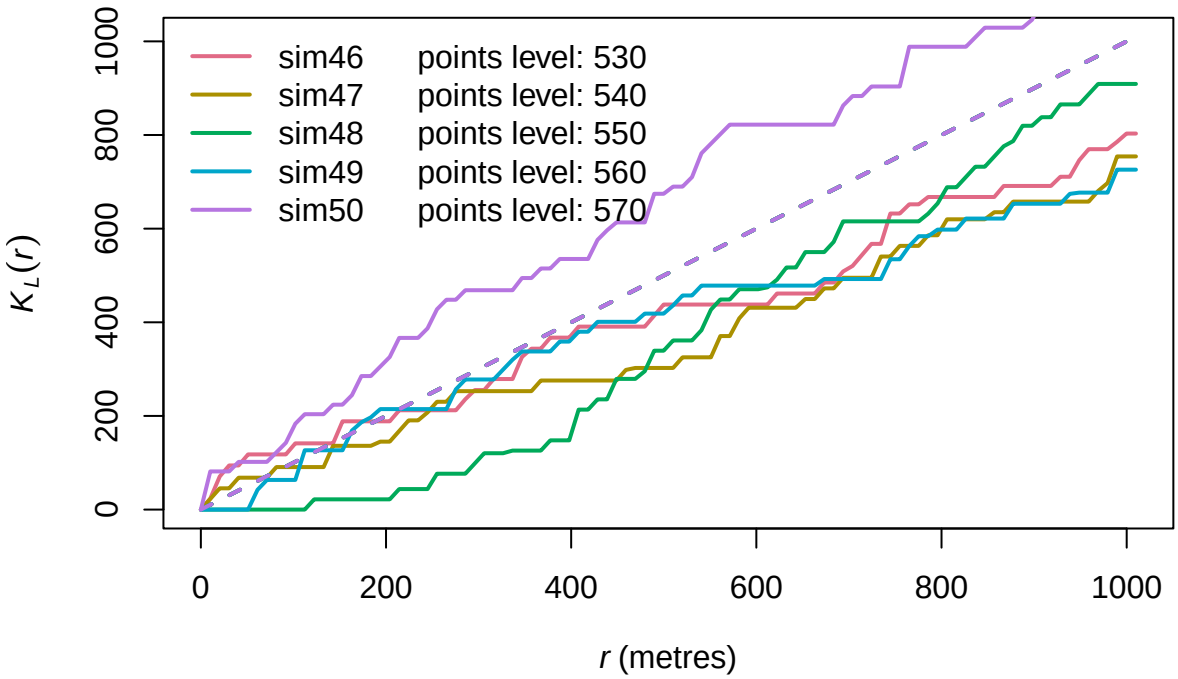

sim21 points level: 280
sim22 points level: 290
sim23 points level: 300
sim24 points level: 310
sim25 points level: 320
$K_L(r)$
0
200
400
600
800
0
200
400
600
800
1000
$r$ (metres)


sim26 points level: 330
sim27 points level: 340
sim28 points level: 350
sim29 points level: 360
sim30 points level: 370
$K_L(r)$
0
200
400
600
800
1000
r (metres)


sim31 points level: 380
sim32 points level: 390
sim33 points level: 400
sim34 points level: 410
sim35 points level: 420
$K_L(r)$
0
200
600
1000
r (metres)


sim36 points level: 430
sim37 points level: 440
sim38 points level: 450
sim39 points level: 460
sim40 points level: 470
$K_L(r)$
0
200
400
600
800
1000
r (metres)

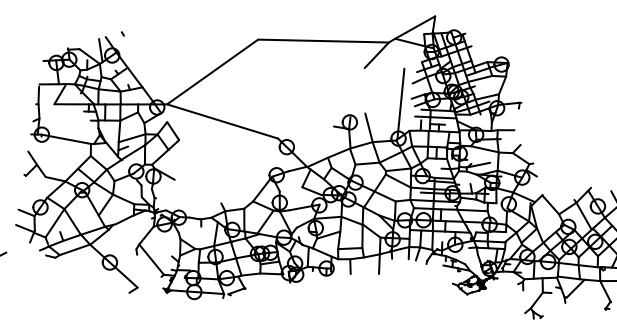

sim41 points level: 480
sim42 points level: 490
sim43 points level: 500
sim44 points level: 510
sim45 points level: 520
$K_L(r)$
0
200
600
1000
0
200
400
600
800
1000
r (metres)




```
for (b in seq(1, B, by = 5)){
  set.seed(n0 + b)
  X_sim   <- runiflpp(n0, L)              #       n0 * b
  plot(X_sim, main = paste0("Points: ", n0 + 10*b))
}
```

**Points: 80**

**Points: 130**

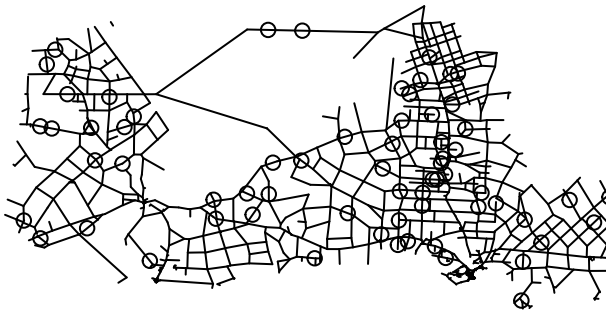


**Points: 180**

**Points: 230**

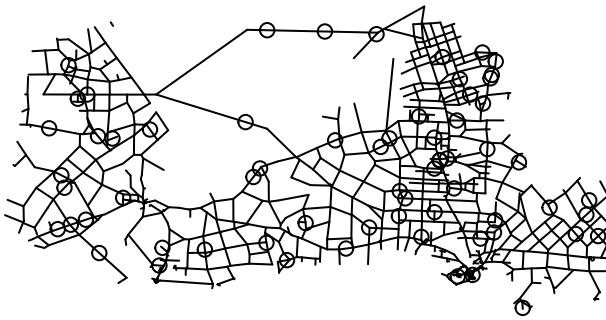
**Points: 280**

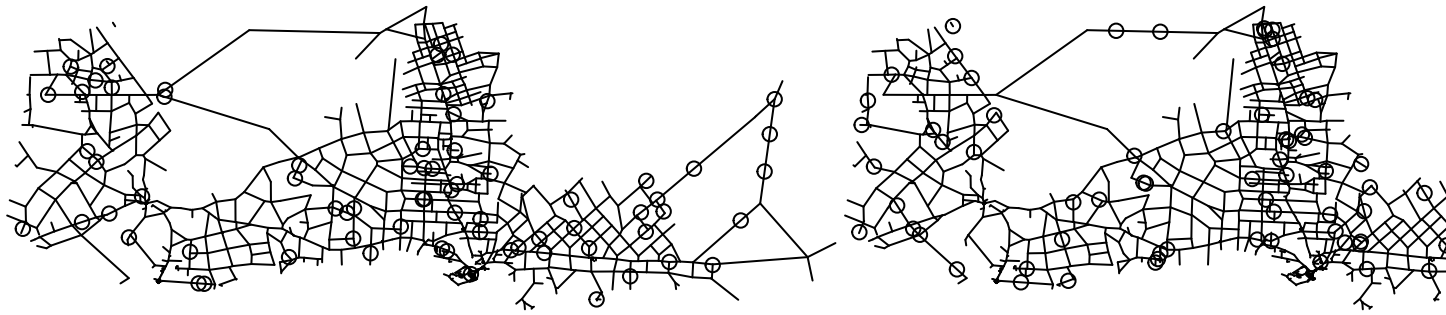
**Points: 330**



**Points: 380**

**Points: 430**






```r
getAnywhere("Knet")
```

```
## A single object matching 'Knet' was found
## It was found in the following places
##   package:spatstat.Knet
##   namespace:spatstat.Knet
## with value
##
## function (X, r = NULL, freq, ..., verbose = FALSE)
## {
##     stopifnot(is.lpp(X))
##     X <- repairNetwork(X)
##     if (missing(r) || is.null(r)) {
##         Mndist <- 0
##         Mxdist <- 2 * mean(nndist(as.ppp(X)))
##         noGrid <- 41
##     }
##     else {
##         stopifnot(is.numeric(r))
##         Mndist <- min(r)
##         Mxdist <- max(r)
##         noGrid <- length(r)
##     }
##     stopifnot(noGrid >= 2)
##     maxR <- Mxdist + (Mxdist/100)
##     nX <- npoints(X)
##     if (missing(freq)) {
##         df <- cbind(coords(X), data.frame(freq = rep(1, nX)))
##     }
##     else {
##         f <- as.integer(freq)
##         if (length(f) == nX & all(f) > 0) {
##             df <- cbind(coords(X), data.frame(freq = f))
##         }
##         else {
##             stop("freq vector is not correctly specified.")
##         }
```

```
##     }
##     Eps <- 1e-04
##     df$tp <- pmax(Eps, pmin(1 - Eps, df$tp))
##     dfGrpd <- aggregate(freq ~ seg + tp, data = df, FUN = "sum")
##     ord <- with(dfGrpd, order(seg, tp))
##     df <- dfGrpd[ord, , drop = FALSE]
##     Seg <- df$seg
##     Tp <- df$tp
##     Freq <- df$freq
##     nX <- nrow(df)
##     L <- as.linnet(X)
##     noVert <- npoints(vertices(L))
##     PL <- as.psp(L)
##     noEdge <- nobjects(PL)
##     EdgeLengths <- lengths_psp(PL)
##     Vert1 <- L$from
##     Vert2 <- L$to
##     z <- .C("createGraphNet", no_of_vertices = as.integer(noVert),
##         no_of_edges = as.integer(noEdge), no_of_crashes = as.integer(nX),
##         crash_seg = as.integer(Seg), crash_tp = as.double(Tp),
##         crash_freq = as.integer(Freq), vert_id1 = as.integer(Vert1),
##         vert_id2 = as.integer(Vert2), edge_length = as.double(EdgeLengths),
##         MAX_Distance = as.double(Mxdist), MIN_Distance = as.double(Mndist),
##         no_of_distance = as.integer(noGrid), max_r = as.double(maxR),
##         verboseIter = as.integer(verbose), kvalue = as.double(numeric(noGrid)),
##         PACKAGE = "spatstat.Knet")
##     r <- seq(0, maxR, length.out = noGrid)
##     df <- data.frame(r = r, theo = r, est = z$kvalue)
##     result <- fv(df, argu = "r", ylab = quote(K[L](r)), valu = "est",
##         labl = c("r", "{%s[%s]^{pois}}(r)", "{hat(%s)[%s]}(r)"),
##         desc = c("distance argument r", "theoretical Poisson %s",
##             "estimate of %s"), fname = c("K", "L"), fmla = . ~
##             r, unitname = unitname(X))
##     return(result)
## }
## <bytecode: 0x11a5df350>
## <environment: namespace:spatstat.Knet>
```